Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Information Technology

SomaiyaSat: Testing the Autonomous Router

Course: Software Testing and Quality Assurance (STQA) | Lab CA : Poster Presentation

Vedant Walunj (16010423114) | Vedant Singh (16010423111)
Ayush Tiwari (16010423124)
Class: LY B.Tech IT | Div: A | Batch: A3

Faculty In-charge:
Prof. Poonam Narkhede/ Dr. Ankita Nagmothe

1. INTRODUCTION

- System under test: the SomaiyaSat mission site and ground station for use case KJS-SRS-01, deployed at somaiyasat.vercel.app.
- Once a second the router scores the onboard queue. A wrong choice can send a mode the link cannot carry, or go silent during a short pass.
- Quality matters because the decision is computed, not animated. The same guard rails can be asserted in a test and read on the dashboard.
- The work applies black-box and white-box testing to that router, its API, and the five public pages a visitor actually opens.

2. PROBLEM STATEMENT

- An autonomous scheduler is unsafe if the learned score can override a dead link, a low battery, or a missed housekeeping frame.
- Defects to catch: a mode below its SNR floor marked feasible, safe mode sending images, a watchdog that never fires, and a command the API accepts when it should not.
- Scope: router policy, the gateway command API, and the deployed UI. Orbit time and the link budget are compressed. This is not a flight-certification campaign.

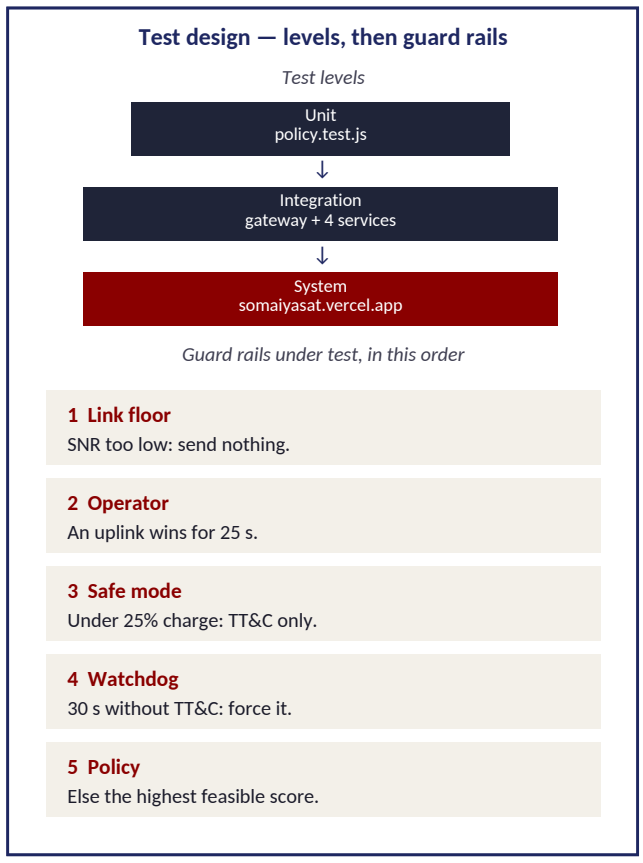
3. OBJECTIVES

- Verify the score: higher-priority TT&C beats SSTV when link and age match, and a mode under its SNR floor is not feasible.
- Verify guard-rail order: link floor, operator override, safe mode, watchdog, then policy. The first match wins.
- Verify the API: health of each service, policy weights, a forced mode, rejection of an unknown command, and reset back to deployment.
- Show the same behaviour on the deployed ground station, so the passing tests match what a user sees.

4. LITERATURE REVIEW

- Strategy: risk-based testing. Guard rails are tested before the score, because a silent spacecraft is worse than a suboptimal frame.
- ISTQB Foundation Level — separate test levels (unit, integration, system) and test types (functional, negative, boundary).
- Myers, Sandler and Badgett — test the boundaries. SNR floors, 25% charge, and 30 s without TT&C are the boundaries that matter here.
- Sommerville — verification asks whether the product meets its specification; validation asks whether the operator can trust what they see.
- Gap: a dashboard can look correct while the policy is wrong. The tests pin the governors; the screenshot only shows that the UI is alive.

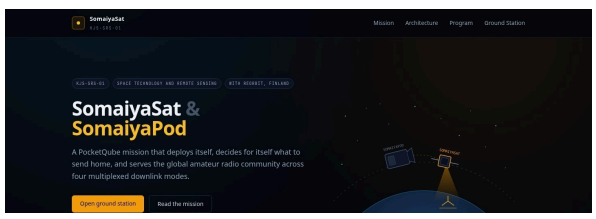
5. METHODOLOGY / SYSTEM DESIGN



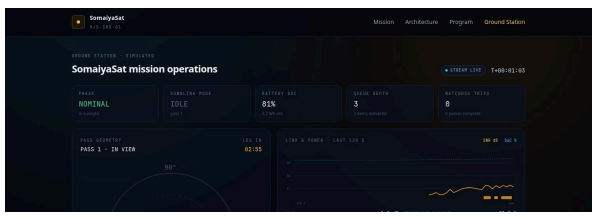
6. IMPLEMENTATION DETAILS

- Tooling. Node's built-in test runner (node --test) with node:assert/strict. No browser harness yet. npm test in microservices/ runs both files.
- White box. policy.test.js calls scoreItem and decide directly: TT&C outscores SSTV, SSTV at 5 dB is infeasible, SNR 0.2 holds the link, safe mode and the watchdog both force TT&C.
- Integration. The test boots orbit, power, queue, router, and the gateway. Health is ok on all four. Priority weight is 0.40. force-mode codec2 is accepted. An unknown command returns HTTP 400. Reset returns phase to deployment.
- System. The deployed landing page and the live telemetry page are the oracle: phase, SNR, battery, queue, and the governor on screen.

7. RESULTS & DISCUSSION



Landing page · system test of the public routes



Ground station · live telemetry oracle

8. KEY FINDINGS

- The score cannot select a waveform below its SNR floor. That case is asserted before any ranking test.
- Safe mode and the watchdog both pre-empt the policy and select TT&C, which is the fail-safe the mission requires.
- The API rejects a command it does not know. A passing happy path is not enough; the negative test is part of the suite.
- What the unit test calls a governor is the same reason string the ground station is built to show.

9. CONCLUSION

- The router is testable. Each guard rail has a case, and the integration test checks the gateway that the UI actually calls.
- Quality here means the scheduler fails safe: no transmission on a dead link, housekeeping only when the battery or the watchdog demands it.
- The deployed site is evidence that those decisions are visible, not only that the pages render.

10. FUTURE SCOPE

- Add route-level UI tests for Home, Mission, Architecture, Program, and Ground Station, and run them in CI.
- Track statement coverage on policy.js and fail the build if a guard rail loses its test.
- Limitation: the pass is time-compressed. These tests check decisions and the API, not a flight link budget.

11. REFERENCES

- [1] ISTQB. Certified Tester Foundation Level Syllabus. Test levels and test types.
- [2] Myers, G.J., Sandler, C. and Badgett, T. The Art of Software Testing. Wiley.
- [3] Sommerville, I. Software Engineering. Pearson. Verification and validation.
- [4] ISO/IEC/IEEE 29119. Software testing — concepts and definitions.
- [5] Node.js. Test runner, node --test. Used by microservices/test.

ACKNOWLEDGEMENT

- Faculty in-charge: Prof. Poonam Narkhede and Dr. Ankita Nagmothe.
- Use-case owners: Dr. Umesh Shinde and Dr. Shailesh Nikam.